> **Draft status:** Working draft, updated 2026-07-14 with the project's first live model result. Every number below comes from actually running the code in this repository — the mock pilot via `scripts/run_benchmark.py`, and a live GPT-4o-mini run via a one-off script noted inline — nothing is projected or extrapolated. Scope is still deliberately modest: 47 task templates and a single live model on 20 tasks, not the full planned 200-task, five-model study. §6.4 lists exactly what's left.

EnterpriseBench

Do Syntactic Tool-Calling Benchmarks Predict Deployment Trustworthiness?

Target venues: - DAI 2026 Industry Track - AAAI 2027

Abstract

Syntactic tool-calling accuracy — did the agent invoke the right function with the right arguments — is the dominant success metric in current LLM agent benchmarks. But enterprise deployment trustworthiness plausibly requires more: does the agent respect organizational policies, stay consistent across multi-turn workflows, and report its own completion state honestly?

We are building EnterpriseBench, a benchmark harness that scores agent tool-use along five dimensions (syntactic accuracy, semantic fidelity, reliability, cost efficiency, latency) plus three deployment-trustworthiness metrics layered on top: Policy Violation Rate (PVR), Multi-Turn Consistency Score (MTCS), and False Completion Rate (FCR). The scoring engine, policy-rule framework, and MTCS/FCR logic are implemented and unit-tested (59 passing tests). The task corpus currently contains 47 hand-written task templates across four verticals (finance, healthcare, legal, DevOps).

We ran two agents against the same 20-task sample suite: a deterministic mock agent (a perfect syntactic agent by construction) and live GPT-4o-mini. Both produced policy violations — the mock agent 1/20 (PVR = 5%), GPT-4o-mini 1/20 (PVR = 5%), the same task and rule in both cases — despite the mock agent matching the ground-truth tool call exactly every time. GPT-4o-mini additionally produced 2 false completions out of 20 (FCR = 10%), a failure mode the mock agent cannot exhibit by construction. This is a real, reproducible first data point consistent with the paper's thesis that syntactic correctness and policy compliance are separable. It is one model on 20 tasks, not yet the broader multi-model study the research question below calls for.

1. Introduction

Enterprise AI deployment is accelerating. Organizations are deploying LLM agents to automate workflows in finance, healthcare, legal, and DevOps — domains where errors are costly, policies are legally binding, and decisions must be auditable. The question practitioners are asking is: which agent can I trust in production?

The research community's answer, so far, is measured largely in syntactic accuracy: did the agent call the right function with the right arguments? Benchmarks like ToolBench (Qin et al., 2024), TAU-bench (Yao et al., 2024), and AgentBench (Liu et al., 2023) define success as matching a reference tool call. This is a reasonable starting point — a wrong function call is clearly a failure — but it doesn't measure everything an enterprise deployer cares about.

Consider three illustrative (not yet empirically demonstrated) scenarios that motivate the project:

- An agent that calls `merge_contacts(source_ids=["C001", "C002"])` could score 1.0 on syntactic accuracy while violating an organizational policy if `C001` has an active deal. Syntactic correctness and policy compliance are independent axes.
- An agent could score well on tool-call matching across many tasks while being inconsistent across a multi-turn workflow — e.g., deciding a ticket's priority is `HIGH` at turn 2 and `LOW` at turn 7 without being asked to change it.
- An agent that reports "task completed" when the underlying system state doesn't reflect that could score 1.0 on self-reported success while masking a real failure — a **false completion**.

EnterpriseBench aims to measure the gap between syntactic benchmark performance and these deployment-relevant properties. What's actually built so far:

- A scoring engine with five dimensions (syntactic, semantic, reliability, cost, latency) plus policy-violation and false-completion scorers, all unit-tested.
- A policy-rule framework (`policy.py`) with 6 deterministic rules spanning tool authorization, argument completeness, PII handling, and two vertical-specific constraints (legal jurisdiction, DevOps environment naming).
- A Multi-Turn Consistency Score (`mtcs.py`) that checks tool-identity and argument carry-through across consecutive turns of a multi-turn task.
- A False Completion Rate scorer (`evaluate.py`) that compares an agent's self-reported success language against post-state verification.
- Two live agent adapters (`agents.py`: `ClaudeAgent`, `OpenAIAgent`); `OpenAIAgent` has now been run live against GPT-4o-mini (§5.2). `ClaudeAgent` has not yet been exercised against a real API key.
- 47 task templates across 4 verticals, generated via `make_suite()`, and a DuckDB persistence layer for run results.

1.1 Research Question

Do high syntactic tool-calling scores predict deployment trustworthiness (low PVR, high MTCS, low FCR) in enterprise workflows?

Still open at the scale that matters (multiple model families, hundreds of tasks), but no longer untested. We now have one real model on 20 tasks: GPT-4o-mini scored 0.942 mean syntactic accuracy while still tripping a policy rule and producing two false completions (§5.2). Combined with the mock-agent result, this is a first genuine data point in favor of the hypothesis that syntactic accuracy and trustworthiness are separable — not a settled finding, since it's one model on one small sample, but no longer purely hypothetical either.

2. Related Work

(Citations carried over from the original draft; not independently re-verified in this pass — flag for a citation check before submission.)

2.1 Tool-Calling and Agent Benchmarks

ToolBench (Qin et al., 2024) evaluates API tool selection across 16,000+ real-world APIs; organizational policy constraints are not modeled. TAU-bench (Yao et al., 2024) introduces multi-turn tool use with a simulated user and measures trajectory match, not policy compliance. AgentBench (Liu et al., 2023) covers eight environments (code, databases, web); policies are not represented as evaluation objects.

2.2 Enterprise AI Benchmarks

WorkArena (Drouin et al., 2024) evaluates 33 tasks on a single platform (ServiceNow) and measures task success, not policy compliance. TheAgentCompany (Xu et al., 2024) covers 175 tasks across a simulated software company; it does not define or measure PVR, MTCS, or FCR.

2.3 What EnterpriseBench Aims to Add

Benchmark	Enterprise tasks	Policy compliance	MTCS	FCR	Cost tracking
ToolBench	No	No	No	No	No
TAU-bench	Partial	No	No	No	No
AgentBench	Partial	No	No	No	No
WorkArena	Yes (1 platform)	No	No	No	No
TheAgentCompany	Yes	No	No	No	No
EnterpriseBench (target)	Yes (4 verticals)	Yes	Yes	Yes	Yes
EnterpriseBench (current)	Yes (4 verticals, 47 templates)	Yes (6 generic rules)	Yes (pairwise-turn only)	Yes	Yes

3. EnterpriseBench Framework

3.1 Task Design (Current State)

`src/enterprisebench/data.py` defines hand-written task templates for four verticals:

Vertical	Templates	Representative tools
Finance	14	`get_stock_price`, `approve_expense`, `reconcile_account`
Healthcare	14	`lookup_patient_record`, `schedule_appointment`
Legal	14	`search_case_law`, `draft_clause`
DevOps	5	`get_deployment_status`, `rollback_service`

That's **47 distinct templates**, not the 200-task suite described in an earlier draft. `make_suite(n, seed)` can generate a suite of any requested size `n` by cycling through templates (`template_index = i // len(VERTICALS)`) and assigning difficulty round-robin (easy/medium/hard) — but beyond 47 tasks it starts repeating template content with different metadata, not presenting genuinely new tasks. Growing the corpus to a true 200-task, non-repeating suite is on the roadmap (§6.4), not done.

Each task carries a `pre_state` and `expected_post_state` (used for FCR verification) and, optionally, a `turns` list for multi-turn variants (`make_multi_turn_task`).

Fixed 2026-07-14: instruction templates previously carried unformatted placeholders (e.g., `"Fetch {service} deployment state..."`) straight into the agent prompt. This was caught by a live smoke test — GPT-4o-mini correctly asked for clarification instead of guessing, scoring zero on every dimension that depends on a tool call, which is what exposed the bug. `make_task()`/`make_multi_turn_task()` now format each instruction against its own `args` dict before handing it to an agent.

3.2 Five-Dimensional Scoring (Implemented, as coded in `evaluate.py`)

Dimension	Formula	Status
Syntactic	`0.4 × name_match + 0.35 × key_jaccard + 0.25 × value_jaccard`	Implemented, tested
Semantic	Fuzzy token overlap on tool name + argument values, with a small instruction-overlap boost	Implemented, tested
Reliability	Fraction of repeated calls matching the expected tool name	Implemented, tested — **but** `evaluate_result()` currently calls it with a single predicted call per task, so in practice it only ever returns 0.0 or 1.0 per run. It hasn't yet been wired up to actually re-run a task multiple times and measure variance across repeats, which is what the metric is meant to capture.
Cost	`max(0, 1 – cost_usd / budget_usd)`, budget = $0.01/task	Implemented, tested
Latency	`max(0, 1 – latency_ms / budget_ms)`, budget = 2000ms/task	Implemented, tested

No bootstrap confidence intervals or paired significance testing are implemented anywhere in the codebase yet. An earlier draft of this paper claimed a `bootstrap_ci` / `paired_bootstrap_test` implementation in `evaluate.py` following Dror et al. (2018); that code does not exist. It's on the roadmap, not done — every number in §5 is a raw sample mean with no uncertainty quantification.

3.3 Deployment Trustworthiness Metrics (Implemented, as coded)

Policy Violation Rate (PVR)

```
PVR = |{tasks with ≥1 policy rule violation}| / |all tasks|
```

`policy.py` implements `PolicyRule(name, description, check_fn)` objects bundled into a `TaskPolicy` per vertical. Currently there are **6 rule functions**, distributed as:

Rule	Applies to	What it checks

`authorized_tool`	all verticals	agent only calls the tool named in the task schema
`required_args`	all verticals	all schema-declared parameters are present in the call
`no_empty_args`	healthcare	at least one argument is supplied
`no_pii`	finance, healthcare	no raw SSN / credit-card patterns in arguments
`jurisdiction_explicit`	legal	jurisdiction argument isn't a wildcard (`*`, `ALL`, `any`, empty)
`production_explicit`	devops	`environment` argument isn't the ambiguous abbreviation `"prod"`

This is a real but modest taxonomy — it does **not** yet cover the "authorization / data handling / communication / retention / escalation" 5-category, 8-rule structure described in an earlier draft, and it does not yet include anything like the `merge_contacts` / active-deal CRM example used to motivate the paper in §1 (that example is illustrative of a target scenario, not something the current 6 rules would catch — there is no CRM vertical or `merge_contacts` tool in the codebase today).

Multi-Turn Consistency Score (MTCS)

```
MTCS = (consecutive-turn transitions with no detected violation) / (total transitions evaluated)
```

`mtcs.py` checks, for each consecutive pair of turns in a multi-turn task: (1) the agent doesn't call a tool outside the one declared in the task schema, and (2) arguments that should carry through unchanged from the previous turn actually do. This is narrower than the "arbitrary key-value decision contradiction across any two turns" framing in an earlier draft — the current implementation only compares *adjacent* turns, not all pairs of dependent decisions.

Limitation surfaced during this pass: the current rule set only penalizes contradictions, not omissions. A no-op agent that returns an empty tool call at every turn scores **MTCS = 1.0**, because neither rule fires when there's nothing to contradict (verified by running a no-op mock agent against a 3-turn finance task — see §5.3). This needs a completeness check before MTCS numbers can be trusted as a consistency signal.

False Completion Rate (FCR)

```
FCR = |{agent claims success AND post-state verification fails}| / |agent claims success|
```

`evaluate.py` implements `claims_success()` (regex match against completion language like "done," "completed," "confirmed," etc.) and `verify_post_state()` (checks the predicted call's arguments against the task's `expected_post_state` key-value constraints). This is implemented and tested, and matches the original design description.

3.4 Statistical Infrastructure

Not yet implemented. `evaluate.py` currently provides only sample means/std/min/max (`aggregate_scores`) and a simple leaderboard sort — no confidence intervals, no significance testing. Dror et al. (2018) remains the intended methodology reference for when this is built.

4. Experimental Setup

4.1 Task Suite (Current)

47 templates across 4 verticals (14/14/14/5, see §3.1). `make_suite(n=20, seed=42)` was used to generate the pilot suite reported in §5: 5 tasks per vertical, split across easy/medium/hard difficulty, no multi-turn tasks in the default sample.

4.2 Agents Evaluated

Two agents have been run against the same 20-task suite (seed=42):

1. A deterministic **mock agent** (`scripts/run_benchmark.py`): always returns the task's ground-truth `expected_call` and `expected_output`, with a hardcoded cost of $0.002/task. This is the logical upper bound on syntactic accuracy, used to isolate whether policy/consistency/completion failures can occur even when tool-calling is perfect.
2. **Live GPT-4o-mini** via the `OpenAIAgent` class in `agents.py` (real OpenAI function-calling, cost computed from actual token usage). Results in §5.2.

`ClaudeAgent` exists with the equivalent Anthropic tool-use wiring but has not yet been run against a live API key. The remaining planned SLM families (Mistral, Meta, Microsoft, Google) have no adapter code yet at all — not just missing keys — and would each need a hosting-provider decision (e.g., Together.ai, Fireworks, Groq) before they could be added.

4.3 Reproducibility

```
# Mock pilot run (no API key needed) — reproduces all numbers in §5.1
python scripts/run_benchmark.py

# Live GPT-4o-mini run — reproduces §5.2. Requires OPENAI_API_KEY.
# (OpenAIAgent from src/enterprisebench/agents.py driven via
#  BenchmarkSuite.run_agent() against make_suite(20, seed=42);
#  not yet wrapped into a standalone script in this repo.)
```

5. Results

5.1 Mock-Agent Pilot: 20-Task Suite, Seed 42

Running `python scripts/run_benchmark.py` produces:

Dimension	Mean	n
Syntactic	1.000	20
Semantic	0.908	20
Reliability	1.000	20
Cost	0.800	20

Latency	1.000	20
False Completion	1.000	20
Policy Violation	0.950	20

FCR = 0% (0/20 tasks: the mock agent's outputs and predicted calls always satisfied post-state verification when it claimed success).

PVR = 5% (1/20 tasks). The single violation: a DevOps task instructing the agent to fetch deployment status for the `prod` environment — the expected call used the abbreviated `"prod"` string, which trips the `production_explicit` rule (the rule wants the environment spelled out, not abbreviated). This is a genuine, reproducible instance of the paper's core claim in miniature: a syntactically perfect call (the mock agent, by construction, always matches `expected_call` exactly) still tripped a policy rule, because the ground-truth expected call and the policy rule were written independently of each other.

This is a single data point from a single deterministic agent on 20 tasks — it demonstrates the measurement pipeline works end-to-end and gives one concrete example of syntactic/policy independence. It is not evidence about how any real LLM behaves.

5.2 Live GPT-4o-mini: Same 20-Task Suite, Seed 42

Running `OpenAIAgent` (model `gpt-4o-mini`) against the identical suite used for the mock pilot produces:

Dimension	Mean	n
Syntactic	0.942	20
Semantic	0.829	20
Reliability	1.000	20
Cost	0.991	20
Latency	0.036	20
False Completion	0.900	20
Policy Violation	0.950	20

Total cost: $0.00182 for 20 tasks.

PVR = 5% (1/20) — the same task and the same rule as the mock-agent run: a DevOps instruction that itself reads "...in **prod** for incident report." GPT-4o-mini faithfully copies `"prod"` into the `environment` argument, tripping `production_explicit`. This is the clearest evidence so far for the paper's thesis: a genuinely capable model, not just the mock agent, reproduces the exact same policy violation, because the ambiguity is written into the instruction text itself rather than being a model failure to reason about policy.

FCR = 10% (2/20) — a failure mode the mock agent cannot exhibit by construction (it always tells the truth about its own state). Two real instances: - A legal task where the model's text claims to have "found relevant precedents concerning breach of contract" with no verified matching tool call behind that claim. - A finance task where the

model claims to have "retrieved the stock prices for GOOGL" while its own output simultaneously admits "there was an issue obtaining the specific..." — the model reports success and failure in the same breath, and the success framing is what `claims_success()` picks up.

Reliability = 1.000 carries the same caveat as the mock run: it's a single-sample metric today, not yet driven by repeated calls against the same task, so it can't yet speak to GPT-4o-mini's run-to-run consistency.

Latency = 0.036 is likely measuring the budget, not the model. The 2000ms budget was set with an instant mock call in mind; live GPT-4o-mini calls involve two network round trips (initial call, then a follow-up for the natural-language summary) and routinely took 3-4+ seconds. Any live model will score poorly here until the budget — or the two-round-trip design — is reconsidered. This number should not yet be read as "GPT-4o-mini is slow" so much as "the latency budget wasn't calibrated for a live agent loop."

Two real bugs were found and fixed while producing this result (see `git log`): unformatted instruction placeholders (§3.1) and a crash in `OpenAIAgent` when a response contained more than one tool call (the API rejects a request that doesn't answer every `tool_call_id`). Both are fixed as of this run.

5.3 MTCS Pilot (Single Task, Illustrative)

Running a no-op mock agent (returns an empty tool call every turn) against a 3-turn finance multi-turn task (`make_multi_turn_task(vertical="finance")`) scores **MTCS = 1.0** (2/2 transitions "passed"). As noted in §3.3, this is a limitation, not a positive result: the current rule set has nothing to say about an agent that does nothing. A real MTCS pilot needs an agent that actually attempts turns, plus a completeness/omission check in `mtcs.py`.

5.4 Cost-Quality Pareto Analysis

Not producible yet — requires results from more than one agent. `pareto_frontier()` is implemented in `evaluate.py` but has not been exercised on real multi-agent data.

6. Discussion

6.1 What the Evidence So Far Actually Shows

Both agents tested — a mock agent that is syntactically perfect by construction, and live GPT-4o-mini, which is highly capable but not perfect (0.942 mean syntactic score) — tripped the identical policy rule on the identical task. That's a meaningfully stronger data point than the mock-only result alone: it shows the same syntactic/policy gap surviving contact with a real model, not just a constructed one. GPT-4o-mini also produced false completions the mock agent structurally cannot. Read this as an encouraging first result, not a settled finding — it's one model family on 20 tasks, and PVR/FCR rates this small (1/20, 2/20) come with wide uncertainty that the benchmark doesn't yet quantify (§3.4).

6.2 What Would Turn This Into a Full Empirical Study

1. Run `ClaudeAgent` live (the OpenAI side is now done) and add the remaining planned SLM families once hosting/API decisions are made, to see whether the PVR/FCR pattern holds across models or is specific to GPT-4o-mini.
2. Grow the task corpus past 47 templates toward something closer to the original 200-task target, with genuinely distinct tasks per vertical rather than repeated templates.
3. Add a completeness/omission check to MTCS so it can't be trivially maxed out by an agent that does nothing (§3.3, §5.3).

4. Implement bootstrap confidence intervals and paired significance testing (§3.4) — at n=20, the PVR and FCR rates reported in §5.2 need error bars before they support any comparative claim.
5. Recalibrate the latency budget for a live, multi-round-trip agent loop rather than an instant mock call (§5.2).
6. Expand the policy rule set toward the richer taxonomy (authorization / data handling / communication / retention / escalation) sketched in the introduction, including a worked CRM-style example if that vertical is added.

6.3 Cost, If and When Live Evaluation Happens

Confirmed live: the total cost of the 20-task GPT-4o-mini run was $0.00182, computed from real token usage via the same pricing formula described in `agents.py`. Cost tracking works as designed.

6.4 Limitations (Honest Accounting)

- **Only one live model family has been evaluated (GPT-4o-mini, one 20-task run).** The research question in §1.1 is meaningfully started but not answered at the scale the paper's title implies.
- **Task corpus is 47 templates, not 200**, and DevOps has only 5 (vs. 14 for the other three verticals) — thin coverage, especially for that vertical.
- **Policy taxonomy is 6 generic rules**, not the 8-rule/5-category structure described conceptually in the introduction; several motivating examples (CRM `merge_contacts`, communication/retention/escalation policies) aren't implemented.
- **No bootstrap CI or significance testing** is implemented; the PVR/FCR rates in §5.2 (1/20, 2/20) are point estimates with no uncertainty quantification — don't over-read the exact percentages yet.
- **MTCS only checks adjacent-turn contradictions, not omissions** — a do-nothing agent scores perfectly (§5.3).
- **Reliability dimension isn't yet exercised as a repeated-run metric** in the benchmark loop — it's implemented but always called with n=1 predicted call today, so it can only return 0 or 1, not a meaningful consistency fraction.
- **The 2000ms latency budget appears miscalibrated for live, multi-round-trip agent calls** (§5.2) — treat latency scores for any live model as suspect until this is revisited.
- **Audit Trail Reconstructibility (ATR)**, mentioned as a target metric, is not implemented and would require human raters.
- **Citations in §2** are carried over from an earlier draft and have not been independently re-verified in this pass.

7. Ethics and Broader Impact

All tasks use synthetic data; no real enterprise PII is involved. Total spend to date is $0.00182 (the GPT-4o-mini run in §5.2); environmental/cost impact remains negligible at this scale. Cost is tracked per-call via the adapter cost computation (§6.3) and will scale linearly and transparently as more models and tasks are added.

8. Conclusion

EnterpriseBench's measurement infrastructure — five-dimensional scoring, a policy-violation engine, MTCS, and FCR — is implemented and unit-tested (59 passing tests) against a 47-template, 4-vertical task corpus. The first live model run (GPT-4o-mini, 20 tasks, $0.00182 total cost) reproduced the same policy violation as the mock-agent pilot despite scoring 0.942 on syntactic accuracy, and additionally surfaced 2 false completions that a mock agent cannot exhibit by construction. That is real, if preliminary, evidence for the paper's central thesis. The next concrete steps are running `ClaudeAgent` and additional SLM families, growing the task corpus, and adding the statistical infrastructure needed to make the PVR/FCR rates in §5.2 defensible at a larger scale rather than a first

data point.

References

Liu et al. (2023). AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688. Drouin et al. (2024). WorkArena: How Capable are Web Agents at Solving Common Knowledge Work Tasks? arXiv:2403.07718. Yao et al. (2024). τ-bench: A Benchmark for Tool-Agent-User Interaction. arXiv:2406.12045. Qin et al. (2024). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. ICLR 2024. Xu et al. (2024). TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. arXiv:2412.14161. Mialon et al. (2023). GAIA: A Benchmark for General AI Assistants. arXiv:2311.12983. Dror et al. (2018). The Hitchhiker's Guide to Testing Statistical Significance in NLP. ACL 2018.

Appendix A: Implementation Status (Verified Against Repo, 2026-07-14)

Component	Status	Location
Task templates (4 verticals, 47 templates)	Implemented; instruction placeholders now formatted correctly	`src/enterprisebench/data.py`
Core data model (`BenchmarkTask`, `BenchmarkSuite`, etc.)	Implemented, tested	`src/enterprisebench/core.py`
5-dimensional scorer	Implemented, tested	`src/enterprisebench/evaluate.py`
Policy taxonomy + PVR (6 rules)	Implemented, tested	`src/enterprisebench/policy.py`
MTCS (adjacent-turn only)	Implemented, tested	`src/enterprisebench/mtcs.py`
FCR (`claims_success`, `verify_post_state`, `score_false_completion`)	Implemented, tested	`src/enterprisebench/evaluate.py`
`OpenAIAgent` adapter	Implemented, tested, **run live** against GPT-4o-mini (§5.2); multi-tool-call handling fixed	`src/enterprisebench/agents.py`
`ClaudeAgent` adapter	Implemented, not yet run live	`src/enterprisebench/agents.py`
DuckDB persistence layer	Implemented	`src/enterprisebench/db.py`
Mock-agent demo/benchmark runner	Runnable	`scripts/run_benchmark.py`
Bootstrap CI + paired significance test	**Not implemented**	—
Live SLM empirical study (5 families)	**1 of 5 done** (GPT-4o-mini); Mistral/Llama/Phi/Gemma have no adapter code yet	—
ATR (Audit Trail Reconstructibility)	**Not implemented**	—

Public `__init__.py` export of policy/MTCS/FCR scorers	**Not exported** — currently only core + 5-dim scorers are in `__all__`	`src/enterprisebench/__init__.py`

Appendix B: Notation

Symbol	Meaning
PVR	Policy Violation Rate
FCR	False Completion Rate
MTCS	Multi-Turn Consistency Score
ATR	Audit Trail Reconstructibility (not implemented)
SLM	Small Language Model